Course Code: CSE 499B
Section: 13
Project Group No: 06
Group Members:
Md.Murad Ul Momin (ID: 2232360642)
Rima Saha (ID: 2132517042)
Sakia Tabassum Ananna (ID: 2221433042)
Ashraful Hossain Riyan (ID: 2232845642)

PRIVATE SHIELD: Weekly update 2

I. INTRODUCTION

This week we extended PRIVATE SHIELD with a prepared-model and prepared-data workflow. The implementation can load compatible saved MLP weights, validate input descriptions, make predictions, evaluate labeled data, and save a reproducible run record. Verification used synthetic inputs and fixed artificial weights without model training. These checks do not establish research results.

II. IMPLEMENTATION WORK

A. Model Loading and Compatibility

checkpoint.py loads validated weights into an existing compatible model. contract.py records architecture, feature order, class meanings, and a preprocessing fingerprint. artifact.py loads a complete model package with its own contract, purpose flags, and file hash. Missing metadata, incompatible tensors, and inconsistent declarations are rejected. Feature and class counts remain configurable rather than fixed to CICIoT2023.

B. Prepared Data Interface

manifest.py reads a JSON description of an already-prepared dataset, resolves file paths, and checks its declared role and preprocessing fingerprint. prepared.py validates NPZ features, ordered names, dimensions, labels, and finite numeric values before constructing PyTorch datasets. Labels are optional only for prediction-role data. These modules do not fit preprocessing, create real client partitions, or infer class mappings.

C. Prediction, Evaluation, and Logging

predict.py returns class IDs, readable labels, and probabilities while preserving row order and ignoring target labels. evaluation/prepared.py compares predictions with supplied labels and reuses overall and per-class metrics, adding mean cross-entropy loss. Neither path updates model weights. prepared_logger.py creates unique folders with settings, metadata, results, and a completion summary. Reproducibility utilities record seeds, versions, device, Git information, and explicit testing-only status.

III. INTEGRATION AND CURRENT STATUS

prepared_runner.py connects configuration, compatible input loading, execution, and logging. main.py adds an explicit prepared workflow alongside dummy FedAvg verification. Two YAML templates select prediction or evaluation. A selected Milestone B regression run recorded 568 passing cases; this was not the full test inventory. Checks covered compatibility, known outputs, label independence, reproducibility, and preservation of previous files. The prepared path supports the current PyTorch MLP format; compatible real inputs remain required.

FUTURE OBJECTIVES

- Supply compatible model artifacts and prepared data with verified contracts and provenance.
- Inspect real data and agree class mappings, client partitions, preprocessing, and validation/test separation.
- Start a small real-data FedAvg pilot after software and data-readiness checks pass; keep research methods deferred.